PROGRAMAÇÃO PARALELA · TRABALHO 3

Bubble sort em MPI por divisão e conquista

Árvore clássica × árvore com balanceamento de carga

Grupo 20 · Gabriel Coelho e Gabriel Cherubini

O PROBLEMA

Dividir um algoritmo quadrático já corta o trabalho

$$N^2 \longrightarrow N^2/L$$

trabalho total com L folhas, cada uma ordenando N/L elementos

Bubble sort custa $\Theta(n^2)$. Cada nó da árvore conquista (ordena, se o pedaço tem até δ elementos) ou divide (manda as metades aos filhos e intercala o que volta).

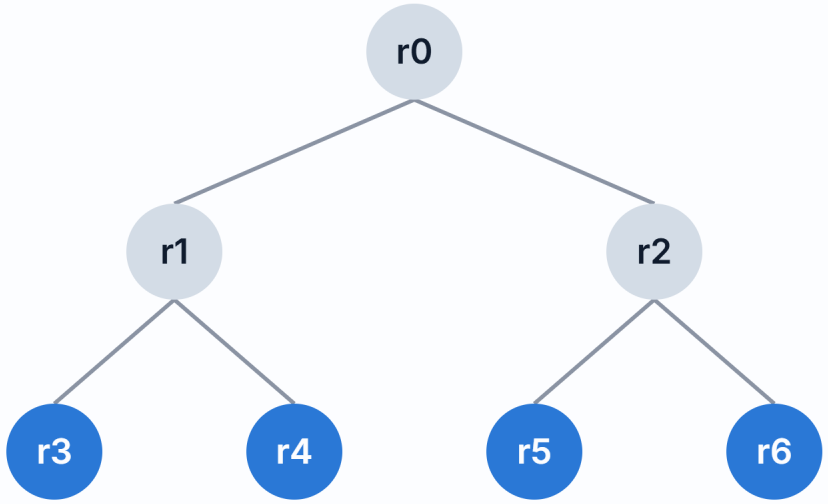
Speed-up acima de p é esperado, não erro de medição: o paralelismo e a divisão somam efeitos.

Entrada: pior caso N, N−1, ..., 1 com N = 10^4, 10^5 e 10^6; a raiz confere se o resultado é 1, ..., N.

IMPLEMENTAÇÃO

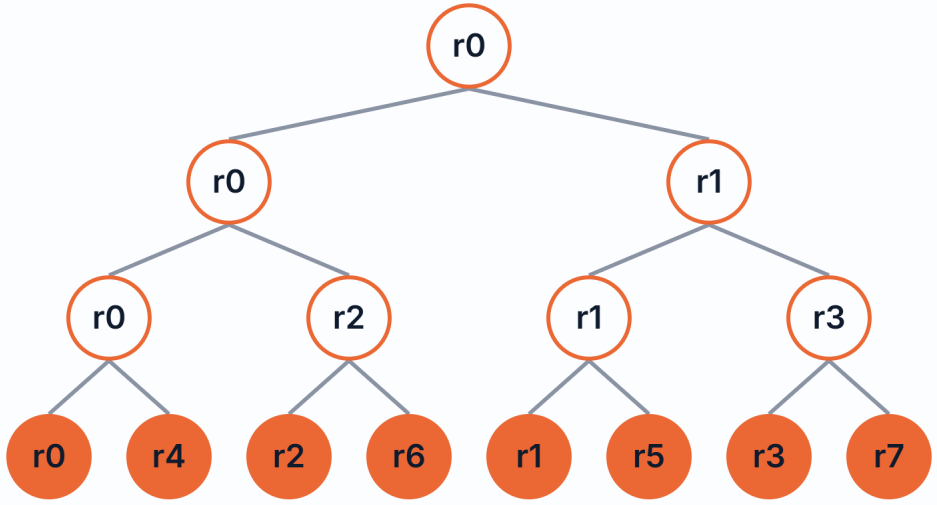
Duas árvores de processos

Árvore clássica (pseudocódigo), p = 7



Filhos 2r+1 e 2r+2. Nós internos (cinza) só dividem e intercalam: **só as (p+1)/2 folhas (azul) ordenam.**

Árvore do artigo (Radenski), p = 8



Cada processo fica com a metade inicial e manda a final a r + 2^nível. **Todos os p processos ordenam (laranja).**

IMPLEMENTAÇÃO

Três decisões que não estavam no enunciado

δ derivado de N e p

δ = ⌈N / ((p+1)/2)⌉ faz a árvore clássica usar exatamente os p processos.

Antes de começar, o programa simula a divisão e aborta se sobrar ou faltar processo, em vez de travar.

MPI_Send, não MPI_Isend

O Open MPI só transfere mensagens grandes quando o remetente volta a chamar o MPI, e o bubble sort não chama.

Com Isend os dois ordenavam em série: **0,202 s contra ≈0,10 s** (N = 20.000, p = 2).

Correções no material dado

Tamanho ímpar perdia um elemento: a metade direita agora leva tam − tam/2.

A intercalação lia vetor[tam] antes de testar o limite e não liberava o vetor auxiliar.

RESULTADOS · SPEED-UP, N = 10^6

Com L folhas, o ideal é S = L^2

Sequencial: **0,098 s · 10,0 s · 993,6 s** (10^4, 10^5, 10^6) · mediana de 3, da Barrier à última intercalação

p	folhas L	ideal L^2	S clássica	S artigo
3	2	4	3,98	3,97
7	4	16	15,5	15,9
15	8	64	60,3	60,9
31	16	256	146	209

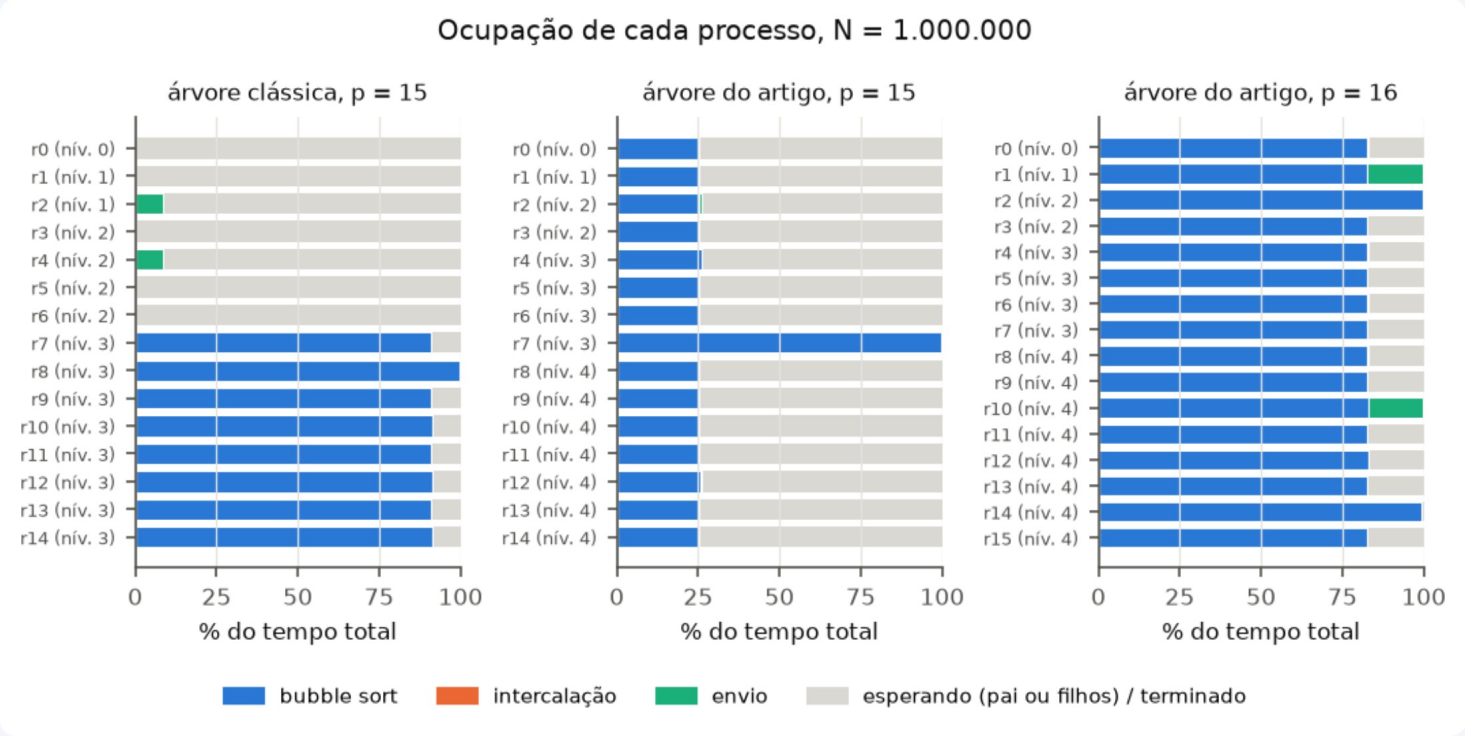
Até 15 processos, quase o ideal: E sobe de 1,33 para 4,02.

Com 31, só 146 de 256: as 16 folhas disputam núcleo com 15 nós internos em espera ativa. Cada folha leva 5,7 s, não 3,9 s.

Com N = 10^4 o custo fixo (~10 ms) domina: E cai de 1,17 para 0,32.

RESULTADOS · BALANCEAMENTO, N = 10^6

Quem trabalha em cada árvore



Clássica, p = 15: 7 nós internos 0% ocupados; esperam 17 s e intercalam em 5 ms. As 8 folhas ficam 92% ocupadas.

Artigo, p = 16: todos ordenam 62.500 elementos em 4,1 s, 83% ocupados.

Artigo, p = 15: o rank 7 não tem ajudante e ordena o dobro (16,3 s); os outros esperam 75% do tempo.

RESULTADOS · MECANISMOS DE BALANCEAMENTO, N = 10^6

A árvore do artigo só compensa com 2^k processos

processos	artigo	clássica
7	62,6 s	64,2 s
8 = 2^3	15,9 s	—
15	16,3 s	16,5 s
16 = 2^4	4,1 s	—
31	4,8 s	6,8 s
32 = 2^5	1,7 s	—

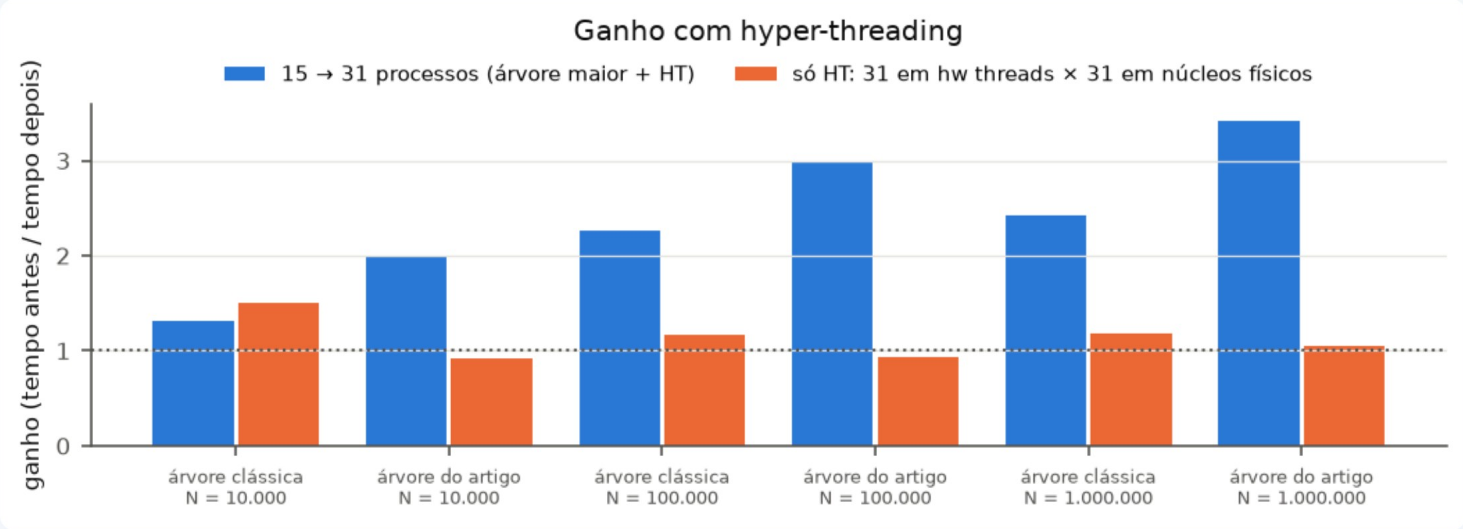
Com 2^k – 1, um rank fica sem ajudante: o artigo empata com a clássica (15: 16,3 × 16,5 s) e com 2^(k–1) processos (7 ≈ 4).

Com 2^k (linhas destacadas), pedaços iguais: $S \approx p^2$ e **4× mais rápido** que a clássica com um processo a menos.

— a clássica só fecha com 2^k – 1 processos.

RESULTADOS · HYPER-THREADING

A maior parte do ganho vem da árvore, não do HT

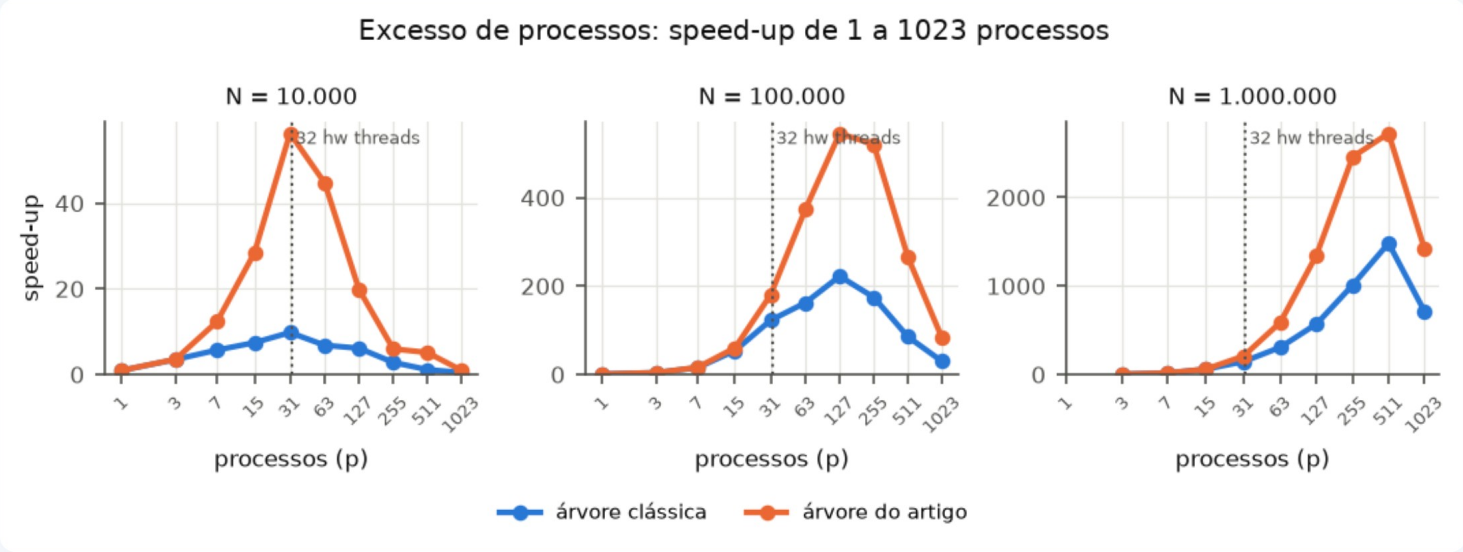


15 → 31 processos (N = 10⁶): 2,4× na clássica, 3,4× no artigo. Mistura dobrar as folhas com o HT.

Só HT (mesmos 31 processos, com e sem as threads extras): 1,18× na clássica, 1,05× no artigo.

RESULTADOS · 63 A 1023 PROCESSOS

Mais processos que threads: o ótimo cresce com N



N = 10^4 · ótimo em **≈31**
1023 fica mais lento que o sequencial

N = 10^5 · ótimo em **127**
S = 222 clássica, 545 artigo

N = 10^6 · ótimo em **511**
S = 1483 clássica, 2724 artigo

CONCLUSÕES

O que os números mostram

Superlinear pelo algoritmo

Dividir o vetor corta o trabalho quadrático: $S \approx L^2$ até as folhas disputarem núcleo.

Clássica desperdiça metade

Os nós internos só esperam: com p processos, só (p+1)/2 ordenam.

Artigo: 4× com 2^k

Todos ordenam pedaços iguais. Com 2^k − 1 um rank fica com o dobro e a vantagem some.

Processos além das threads

Compensam com N grande (até 511 com 10^6); o HT sozinho rende só 5–18%.